Tokenization

Besher Alkurdi

Table of contents

- From Text to Numbers
- Subword Tokenization
- Bytes
- When Tokenization Breaks

From Text to Numbers

What Does a Model See?

The merchant traveled the Silk Road to Damascus

The model sees: `[785, 29238, 30696, 279, 51036, 9536, 311, 57023]`

How did we get here?

Characters

```
"Damascus" →
["D","a","m","a","s","c","u","s"]
```

8 tokens for one word

The merchant traveled the Silk Road to Damascus -> **48 tokens**

No single character encodes word-level meaning. The model must learn that D-a-m-a-s-c-u-s is a city.

Words

```
The merchant traveled the Silk Road to Damascus
```

```
["The", "merchant", "traveled", "the", "Silk", "Road", "to", "Damascus"]
```

8 tokens. Each one meaningful.

The Problem

Training vocabulary: {the, merchant, silk, road, damascus, …}

Inference: The **Damascene** merchant sold spices

Damascene → <UNK>

The model receives no information about the word's form or meaning.

What If?

```
"Damascene" → ["Damasc", "ene"]
"unhappiness" → ["un", "happi", "ness"]
```

Rare words split into known pieces.

Subword Tokenization

BPE

Start with characters. Merge the most frequent pair. Repeat.

Sennrich et al. (2016), *Neural Machine Translation of Rare Words with Subword Units*

BPE: Start

Corpus: `low low low lowest lowest newer newer wider wider`

Split into characters (with end-of-word marker _):

```
l o w _      (×3)
l o w e s t _  (×2)
n e w e r _   (×2)
w i d e r _   (×2)
```

BPE: Merge 1

Count adjacent pairs:

```
(l, o): 5    (o, w): 5    (w, _): 3
(e, r): 4   (r, _): 4    (w, e): 4
```

Most frequent: `(l, o)` → merge into **lo**

```
lo w _        (×3)
lo w e s t _  (×2)
n e w e r _   (×2)
w i d e r _   (×2)
```

BPE: Merge 2

Most frequent: `(lo, w)`: 5

→ **low**

```
low _        (×3)
low e s t _  (×2)
n e w e r _  (×2)
w i d e r _  (×2)
```

Merge 3: `(e, r)`: 4 → **er**

```
low _        (×3)
low e s t _  (×2)
n e w er _   (×2)
w i d er _   (×2)
```

BPE: Merge 4

`(er, _)`: 4 → **er_**

```
low _        (×3)
low e s t _  (×2)
n e w er_    (×2)
w i d er_    (×2)
```

After more merges: `low_`, `low`, `er_`, `est_`, `new`, `newer_`, …

Stop at desired vocabulary size (32K, 50K, 100K).

BPE: At Inference

Apply the **same merges** in the **same order**.

The merge list **is** the tokenizer.

BPE: Applying Merges

New word at inference: **`lowest`**. Start from characters, apply learned merges in order.

CHARACTERS

`l` `o` `w` `e` `s` `t`

↓ merge #1: l + o → lo

`lo` `w` `e` `s` `t`

BPE: Applying Merges (cont.)

lo w e s t

↓ merge #2: lo + w → low

low e s t

↓ merge #5: e + st → est (no merge #3, #4 match)

low est

Final tokens: **low** + **est** - the model understands both pieces.

WordPiece

Same idea, different merge criterion:

BPE: merge the most **frequent** pair

WordPiece: merge the pair that maximizes **likelihood**

$$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a) \cdot \text{freq}(b)}$$

Used in BERT (30,522 tokens). Continuation pieces prefixed with ##.

```
"playing" → ["play", "##ing"]
```

Schuster and Nakajima (2012)

Unigram

BPE builds **bottom-up**: start small, merge.

Unigram builds **top-down**: start large, prune.

Multiple valid segmentations exist → sample different ones during training → the same text is split differently each epoch.

Used in T5, XLNet (via SentencePiece).

Kudo (2018), “Subword Regularization”

Bytes

256

Standard BPE starts from characters. But which characters?

Unicode has **150,000+** codepoints.

Every string in every language is a sequence of bytes (0-255).

Base vocabulary: **256 entries**. Covers everything.

Then apply BPE merges on top.

No UNK tokens, regardless of input.

Radford et al. (2019), GPT-2

The Tradeoff

English: *Hello* → 1 token

UTF-8: each non-Latin character = 2-4 bytes

If the tokenizer hasn’t learned to merge those bytes, each character fragments into multiple tokens.

Tokenization Fertility

Same meaning, different token counts:

Language	*Hello*	Tokens (GPT-2)
English	Hello	1
French	Bonjour	1-2
Russian	Привет	3-6
Japanese	こんにちは	3-5
Thai	สวัสดี	6-10

Same content. More compute. Less context. **Higher API cost.**

Petrov et al. (2023), *Language Model Tokenizers Introduce Unfairness Between Languages*

Tokenization Unfairness

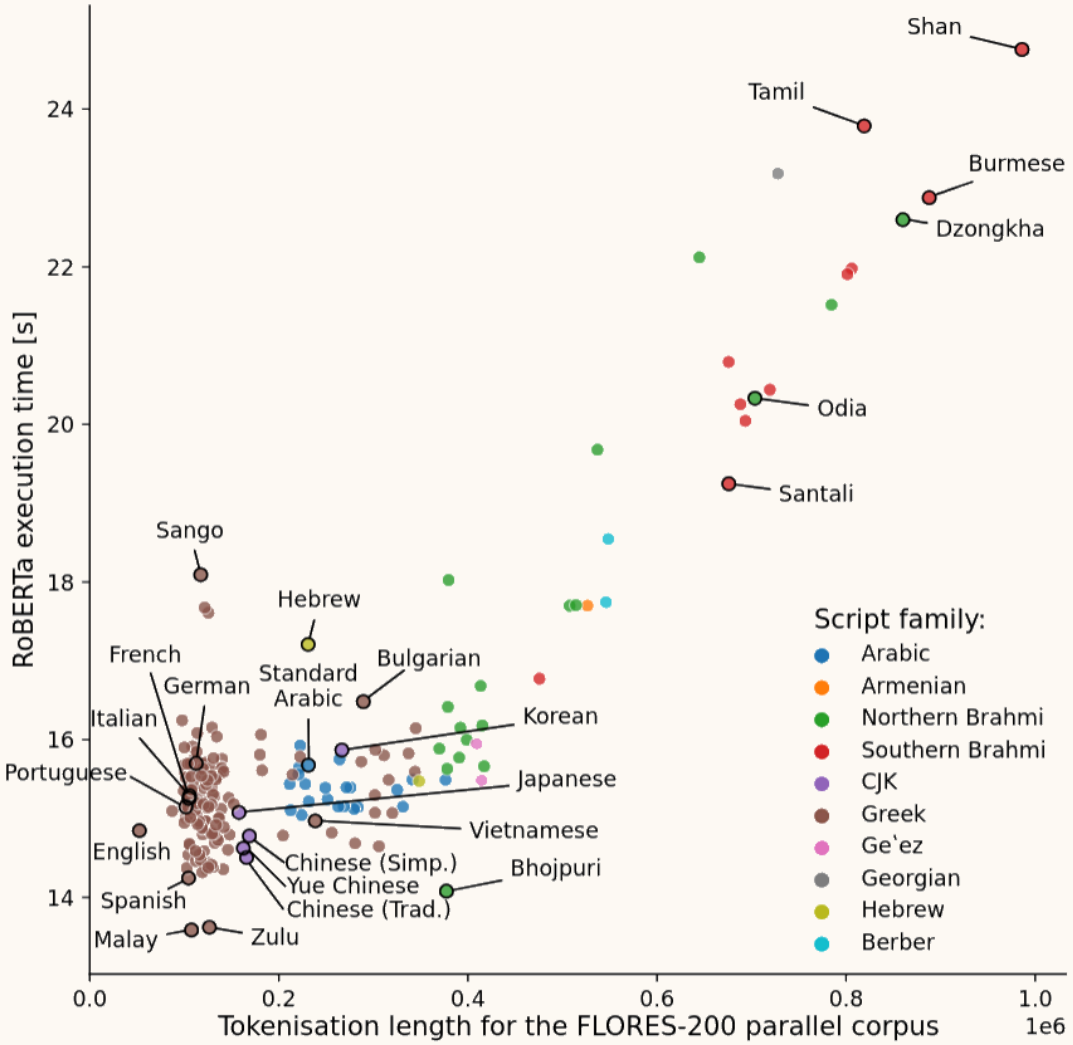


Figure: Petrov et al. (2023)

What Models Use

Model	Vocab Size	Type
BERT	30,522	WordPiece
GPT-2 / 3	50,257	Byte-level BPE
LLaMA 1, 2	32,000	SentencePiece BPE
GPT-4	100,256	Byte-level BPE
LLaMA 3	128,256	Byte-level BPE
Gemma	256,000	SentencePiece

Trend: vocabularies are growing.

Try It

```python
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("gpt2")
tok.tokenize("The Damascene merchant sold spices")
# ['The', 'ĠDam', 'asc', 'ene', 'Ġmerchant', 'Ġsold', 'Ġsp', 'ices']
```

Ġ = leading space in byte-level BPE

Try It

Tiktokenizer

gpt-4o

System | You are an expert in Geopolitics

User | Content

Add message

```
<|im_start|>system<|im_sep|>You are an expert in Geopolitics<|
im_end|><|im_start|>user<|im_sep|><|im_end|><|im_start|
>assistant<|im_sep|>
```

Token count

20

```
<|im_start|>system<|im_sep|>You·are·an·expert·in·Geopo
litics<|im_end|><|im_start|>user<|im_sep|><|im_end|><|
im_start|>assistant<|im_sep|>
```

```
200264, 17360, 200266, 3575, 553, 448, 8333, 306, 505
3, 467, 29744, 1541, 200265, 200264, 1428, 200266, 200
265, 200264, 173781, 200266
```

tiktokenizer.vercel.app

When Tokenization Breaks

A Subtle Bug

You fine-tune a model with this chat template:

```
User: What is the capital of Syria?
```

The backend team deploys it with:

```
User:What is the capital of Syria?
```

Why?

```
"User: What"  →  [User]  [:]  [ What]
"User:What"   →  [User]  [:]  [What]
```

`[ What]` ≠ `[What]`

The model learned embeddings for `[ What]` (with space), not `[What]`. Most tokens in the question now have different representations than during training.

Glitch Tokens

Ask GPT-3 to repeat **SolidGoldMagikarp**:

It responds with **distribute**.

Ask it to repeat **StreamerBot**:

It responds with **You're a jerk.**

Rumbelow and watkins (2023), "SolidGoldMagikarp"

Glitch Tokens in Action

What does the string " SolidGoldMagikarp" refer to?

The word "distribute" refers to the act of distributing or spreading something out among a group of people or things. It can also refer to the way in which something is distributed or arranged. For example, a company may distribute its products to different retailers, or a teacher may distribute assignments to students. Additionally, it can mean to give out something in an orderly way, such as distributing flyers or pamphlets.

Rumbelow and watkins (2023)

Why Do Glitch Tokens Exist?

The tokenizer is trained on **dataset A** (e.g. Reddit, web scrapes).

The model is trained on **dataset B** (curated, filtered).

Tokens that appear in A but not in B have a vocabulary entry but **no learned embedding**.

`SolidGoldMagikarp` was a Reddit username. Frequent enough for BPE to merge it. Absent from model training.

Finding Undertrained Tokens

Embedding norm: undertrained tokens have unusually small (or large) L2 norms - barely moved from initialization.

Cosine distance: their embeddings are far from all other tokens - they live in empty regions of embedding space.

Prompting: ask the model to repeat the token. If output probability is near zero, the token is undertrained.

Land and Bartolo (2024), *Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models*

Under-trained Token Indicators

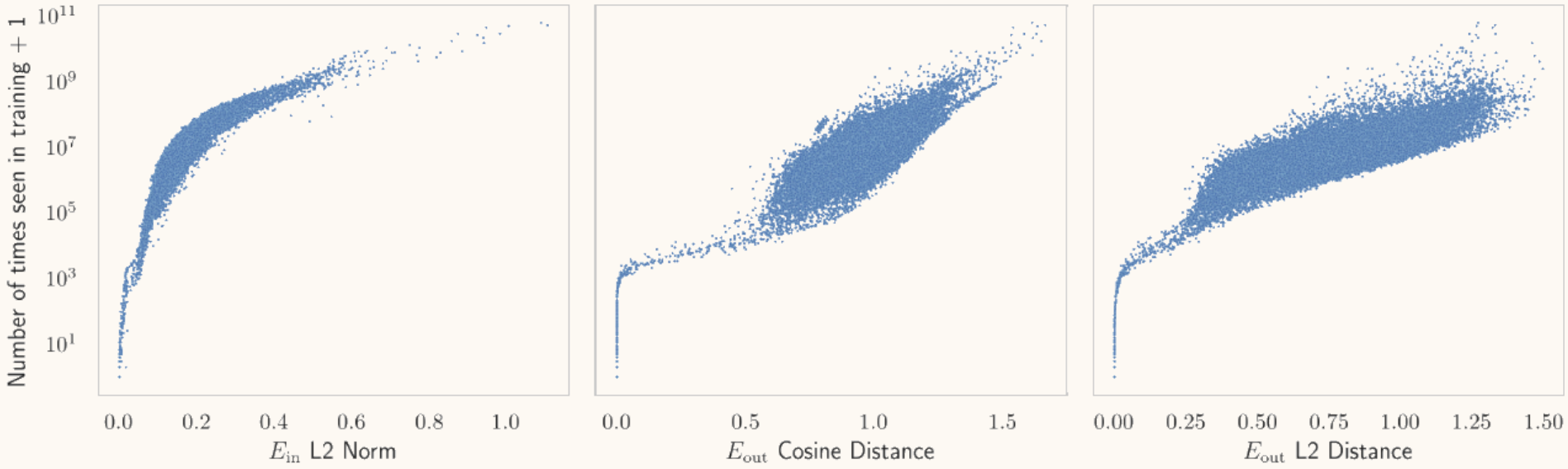


Figure: Land and Bartolo (2024)

What’s Next?

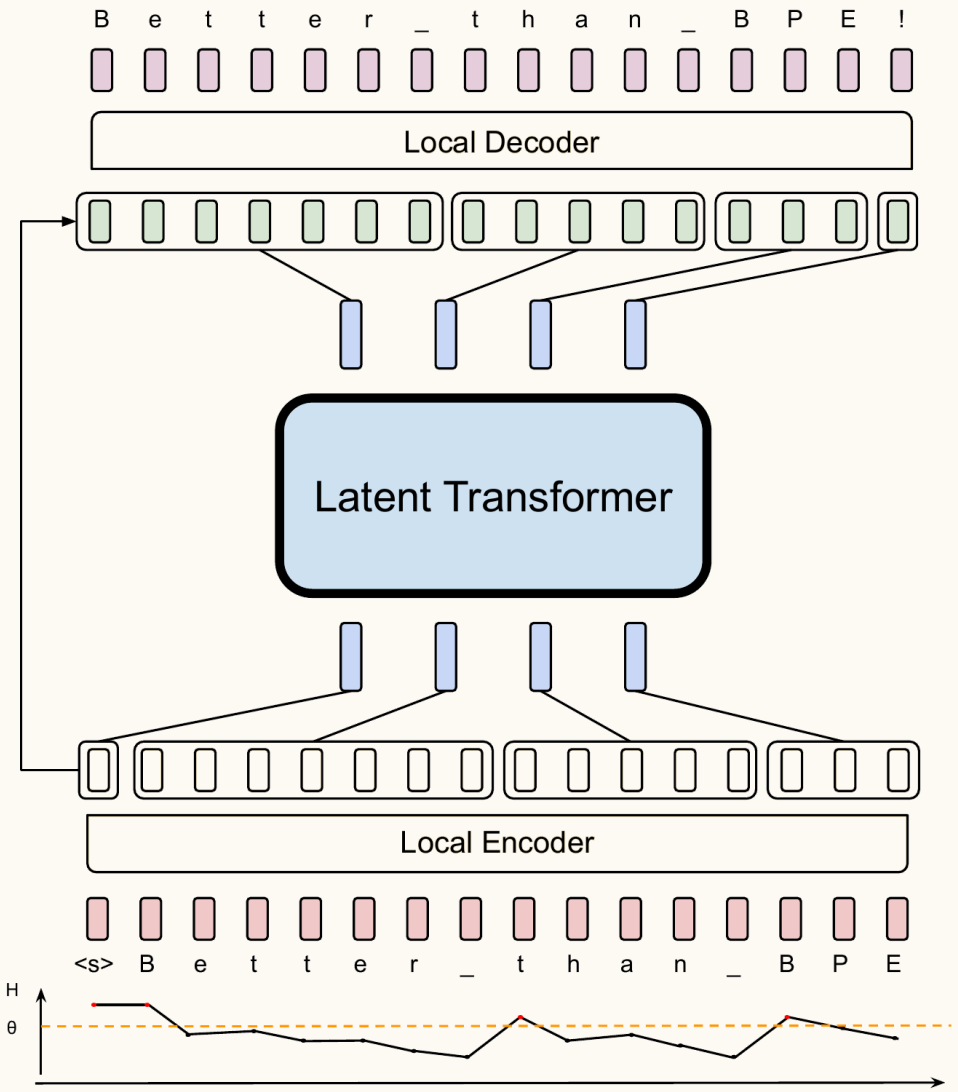
Can we skip tokenization entirely?

Byte Latent Transformer (Meta, 2024): process raw bytes, dynamically group them based on complexity.

Matched LLaMA 3 at 8B scale.

Pagnoni et al. (2024), *Byte Latent Transformer: Patches Scale Better Than Tokens*

BLT Architecture



5. Small Byte-Level Transformer Makes **Next-Byte Prediction**

4. **Unpatching** to Byte Sequence via Cross-Attn

3. Large Latent Transformer **Predicts Next Patch**

2. Entropy-Based Grouping of Bytes Into **Patches** via Cross-Attn

1. Byte-Level Small Transformer Encodes **Byte Stream**

Figure: Pagnoni et al. (2024)

Patching Strategies

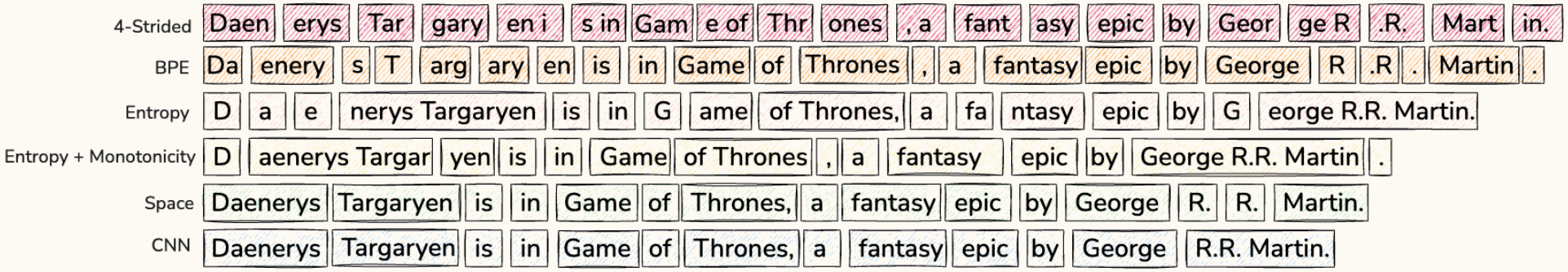


Figure: Pagnoni et al. (2024)

Summary

- Characters → too long, no meaning
- Words → unknown words, huge vocabulary
- **Subwords** → the standard since 2016
  - BPE (GPT), WordPiece (BERT), Unigram (T5)
- Byte-level BPE → no UNK tokens, ever
- Tokenization is fragile: whitespace, glitch tokens, undertrained embeddings